# 分组项目 · 从需求到交付 · 标准打法参考案例(C/S 版)

> **示范项目**:图书馆借阅系统 **技术栈**:Java 11 + Swing(客户端界面)+ Socket(自定义文本协议)+ JDBC(服务端数据访问)+ MySQL。IDE 用 IntelliJ IDEA。**不引入任何第三方框架**(连 JSON 都是手写的极简序列化)。 **用途**:C/S 分组项目的"黄金样板"。各组照这套**打法**走完自己选的题目;每个环节给 **① 多步提示词 ② 预期产出 ③ 验收 checklist ④ 常见踩坑 + 纠错提示词**。 **分工模型**:按业务模块横切 + 契约先行。角色见下。 **状态**:✅ 全 8 环节(0-7)+ 5 个模板已铺开。**环节 2-3 的数据字典与通信协议已逐条对照答案版工程核对一致**。答案版工程:`05-projects/cs-swing-library-demo/`(共享 common + 服务端 server + 客户端 client 三段式,已跑通:登录按角色分流 / 图书查询联表带出 categoryName / 借书事务防超借 / 还书库存回补 / 续借延期 / 管理员图书增删改 / 借阅记录与超期统计)。

---

## 这套打法要治的三个病

| 病 | 根因 | 本案例的制度性解法 |
|---|---|---|
| 组员工作交叉、互相影响 | 分工没有"边界物" | **契约先行**:先锁数据库 schema + 通信协议契约,把它当边界;服务端按命令组 + 命名前缀隔离;客户端只有 1 人不抢文件 |
| 数据库设计不细,AI 服务端/客户端反复踩坑 | 边写边改表 | **数据字典先行**:动代码前先出逐字段数据字典 + 关系 + 建库 SQL,过评审 checklist,锁进 `.trae/rules` 当**单一事实源** |
| 效果出来后不会局部调整 | 只会整段重写 | 环节 6 专教:定位法 + 纠错三步法 + **局部调整场景提示词库** |

---

## 角色与责任矩阵(5-6 人 · 横切)

| 角色 | 人数 | 职责 |
|---|---|---|
| 组长 | 1 | 整体设计(需求/数据库/协议主导)+ 每半天集成 + 交付把关 + 维护共享基建 |
| 服务端 | 2 | 按命令组切,各领一半,命令字 + service/dao 命名前缀隔离 |
| 客户端 | 1 | 全部 Swing 窗口,按契约先行(用假数据/桩先行,不等服务端) |
| 测试 | 1 | 全程挑刺 + 主导系统测试 |
| 文档 | 1 | PPT + 说明文档,兼**协议/数据字典书记员** |

**★主责 / ○参与**

| 环节 | 组长 | 服务端甲 | 服务端乙 | 客户端 | 测试 | 文档 |
|---|---|---|---|---|---|---|
| 0 分工协作规则 | ★ | ○ | ○ | ○ | ○ | ○ |
| 1 需求 + PRD | ★ | ○ | ○ | ○ | ○ | ○成文 |
| **2 数据字典(锁 schema)** | ★ | ○ | ○ | | ○评审 | ○维护 |
| **3 通信协议(锁契约)** | ★ | ○定协议 | ○定协议 | ○确认 | ○评审 | ○维护 |
| 4 分模块开发 | ○集成 | ★登录+查询 | ★借还+管理 | ★所有窗口 | ○单命令测 | ○写说明 |
| 5 联调 + 测试 | ○ | ○修 | ○修 | ○修 | ★主导 | ○ |
| 6 局部调整/迭代 | ○把关 | ★ | ★ | ★ | ○回归 | ○ |
| 7 审查+上线+交付 | ★把关 | ○ | ○ | ○ | ○回归 | ★手册/PPT |

**服务端 2 人切法**(按命令组,不按层;各只碰自己命令字的 case 分支 + 自己前缀的 service/dao 文件):

| Owner | 命令字(共 13 条) | 只碰的服务端文件 |
|---|---|---|
| 服务端甲 | `LOGIN` `REGISTER` `BOOK_QUERY` `CATEGORY_LIST` | `UserService` / `UserDao` 、`CategoryDao` 、`BookService.query` / `BookDao.query` ;负责 `ClientHandler` 里这 4 个 case 分支 |
| 服务端乙 | `BORROW` `RETURN` `RENEW` `MY_BORROW` `BOOK_ADD` `BOOK_UPDATE` `BOOK_DELETE` `READER_LIST` `BORROW_RECORD` | `BorrowService` / `BorrowDao` 、`BookService.add/update/delete` / `BookDao.insert/update/delete` ;负责 `ClientHandler` 里这 9 个 case 分支 |
| 客户端 | 全部窗口 | `client/ui/**` 各窗口 + `client/net/Api.java` 协议封装(唯一客户端,不抢) |
| 组长 | 共享基建 | `common/Protocol.java` 、`common/util/Json.java` 、`common/entity/**` 、`server/util/DBUtil.java` 、`server/ServerContext.java` 、`server/LibraryServer.java` 、`ClientHandler` 的 switch 骨架、`.trae/rules` 、`sql/init.sql` (**共享地基只有组长动**) |

> **两处必须写进契约的共享点**(横切能并行的前提,像交接单一样白纸黑字):
>
> 1. **`ClientHandler` 是一份共享文件**:switch 骨架由组长起好,甲乙各自往里**加自己命令组的 case 分支 + 对应 `doXxx()` 方法**,互不覆盖对方的分支;组长每半天合并。
> 2. **`UserDao` / `BookDao` 存在跨人复用**:`READER_LIST`(乙的命令)要读甲的 `UserDao.listReaders()` ;`BookDao` 的 `query()`(甲)与 `insert/update/delete()`(乙)在同一个 DAO 里按方法切。约定:**方法归谁写谁维护,另一人只读调用、不改对方方法签名**——这就是"甲写、乙只读"的交接线。

---

# 🔒 通用护栏:管住 AI 别做多余的事(贯穿所有环节)

> **这是本案例最重要的一条纪律**。AI 编程工具(Trae、Claude Code 等)有个通病:**特别爱做你没让它做的事**——你让它加个字段,它顺手重构了整个 Service;你让它写借书处理,它自作主张把登录、分页、缓存都加上;你让它改一行,它把整个文件重写。弱学生看不懂它多做了啥,一旦它乱动别人的命令分支,横切分工立刻崩。
>
> **所以:每一次对话都要给它套护栏。** 用下面两层机制,双保险。

## 第 1 层 · 常驻护栏(项目一开始就写进 `.trae/rules`,全程生效)

```
## 全局铁律(每次生成都必须遵守)
1. 只做我当前这一步明确要求的事;做完就停,等我确认再继续。
2. 不要新建我没提到的文件、表、命令、依赖、配置。
3. 不要"顺手"优化、美化、重命名或重构任何已经存在的代码。
4. 不要改动本次任务范围之外的任何文件(尤其别人负责的命令分支/模块)。
5. 不要自作主张加功能(登录、分页、缓存、导出等我没要求的一律不加)。
6. 不确定的地方先问我,不要猜着做。
7. 每次改动后,用一句话告诉我"你改了哪几个文件、加了什么",方便我核对。
```

## 第 2 层 · 每条提示词的护栏尾巴(逐次强化,粘在每个 prompt 末尾)

```
【边界】只做上面这一件事,不要动其它文件/模块,不要加我没提到的功能;做完停下,并列出你改了哪些文件。
```

> **本案例下文各环节的多步提示词,为省篇幅没有逐条重复这段【边界】,但你实际发给 AI 时,每一条都要带上**(或至少确认第 1 层常驻规则已生效)。这不是可选项,是纪律。

## 它多做了怎么办(发现越界立即回滚)

```
你这次做了我没要求的改动:【具体指出,比如"你还改了 BorrowService"】。
请撤销这些多余改动,只保留我要求的【那一处】。以后没让你动的文件不要碰。
```

## 讲师现场强调

- 每次让 AI 干活前,先想清楚"这一步只要它做哪一件事",说死范围再发。
- 它一次给一大堆,别急着接受——**先看它的"改动清单",越界的当场让它撤**。
- 这正是范式 2 的核心:你是**指挥和把关的人**,不是被 AI 牵着走的人。

---

# 环节 0 · 组队 · 分工 · 协作规则

> **本环节主责**:组长;全员参与。**在动需求和代码之前,先把"谁干什么、文件归谁、怎么合到一起"说死**——这是防"工作交叉"的第一道闸。 **产出**:一张分工看板(模板见 `templates/分工看板模板.md`)。

## 1. 按责任矩阵认领角色

对着开头的「角色 × 环节责任矩阵」认领:组长 1 / 服务端 2(甲乙)/ 客户端 1 / 测试 1 / 文档 1。5 人组按下方伸缩规则合并(测试兼文档,服务端保留 2 人)。**当场把名字填进分工看板**。

## 2. 定"命令/文件归属"(防交叉的核心)

服务端 2 人按命令组 + 命名前缀隔离,各自只碰自己的分支/文件:

| Owner | 命令字 | 只碰的服务端文件 |
| --- | --- | --- |
| 服务端甲 | `LOGIN` `REGISTER` `BOOK_QUERY` `CATEGORY_LIST` | `UserService` / `UserDao`、`CategoryDao`、`BookDao.query`;`ClientHandler` 里这 4 个 case |
| 服务端乙 | `BORROW` `RETURN` `RENEW` `MY_BORROW` `BOOK_ADD/UPDATE/DELETE` `READER_LIST` `BORROW_RECORD` | `BorrowService` / `BorrowDao`、`BookDao.insert/update/delete`;`ClientHandler` 里这 9 个 case |
| 客户端 | 全部窗口 | `client/ui/**` + `client/net/Api.java` |
| 组长 | 共享基建 | `Protocol` / `Json` / `entity` / `DBUtil` / `ServerContext` / `LibraryServer` / `ClientHandler` 骨架/ `.trae/rules` / `init.sql` |

> 铁律:**共享基建(协议常量、Json 工具、实体、DBUtil、监听主类、ClientHandler 骨架、数据库脚本、`.trae/rules`)只有组长维护**;成员要改,提给组长,别各自改一套(尤其 `Protocol.java` 命令常量,改一个字两端全崩)。

## 3. 定协作与集成方式(弱队友好版)

- **一份主工程,组长托管**:不强求人人会 git。成员各自开发"自己命令组/窗口的文件清单",组长每半天把成员的文件**合并进主工程**跑一次(尤其 `ClientHandler` 的 case 分支要手工合到一个 switch 里)。(会 git 的组可用"一人一分支 + 组长合并"。)
- **集成节奏**:每半天结束前 30 分钟 = 组长集成时间——合代码 → 起服务端 → 起客户端 → 冒烟点一遍 → 记问题。**不许攒到最后一天才合**(攒到最后必崩)。
- **每日站会 3 句话**(每人):我昨天做完了 X / 今天做 Y / 卡在 Z 需要谁配合。

## 4. 冻结纪律(承上启下)

环节 2 锁的数据库、环节 3 锁的通信协议,**一旦冻结,不许私自改**;要改走"变更评审"(组长 + 相关人过一遍,同步改 `.trae/rules` 和 `Protocol.java`)。这是让大家能并行的前提。

## ✅ 环节 0 验收 checklist

- [ ] 分工看板填好:每人角色 + 负责命令组/窗口 + 文件归属

- [ ] 共享基建的唯一 owner(组长)已明确
- [ ] `ClientHandler` 与 `UserDao`/`BookDao` 的**共享交接线**写进看板(谁加哪些 case、谁写哪个方法)
- [ ] 集成方式和节奏(每半天集成)约定好
- [ ] 全组认同"数据库/协议冻结后不私改"

---

# 环节 1 · 需求梳理 + Agent PRD

> **本环节主责**:组长牵头,全员讨论,文档成文,测试挑歧义。 **为什么关键**:学生最常见的翻车是"需求没想清就让 AI 开干",做到一半发现要的不是这个。**先用反向提问把需求逼清楚,写成 Agent PRD(给 AI 的设计图纸),存进 `.trae/rules`**——后面所有环节的 AI 生成都以它为准。 **产出**:`.trae/rules`(项目 PRD + §通用护栏的常驻铁律 + 数据模型占位[环节2填] + 协议占位[环节3填])。

## 多步提示词

### Step 1 · 反向提问梳理需求

```
我要做一个图书馆借阅系统,C/S 结构(Java Swing 客户端 + Socket + 服务端 JDBC 连 MySQL),但需求还没完全想清楚。请你像产品经理一样,每次只问我一个问题,一共问 6-8 个,把需求理清楚:角色有哪些、图书要哪些信息、分类怎么分、借还流程是什么、有没有借阅上限和续借、超期怎么算、界面要哪些窗口。我回答完你再问下一个。
【边界】只提问,不要写任何代码或文档;等我全部答完再汇总。
```

> 讲师带全组一起答。典型答案沉淀:角色=读者(查/借/还/续借/看我的借阅)+ 管理员(管图书、看读者、查借阅记录与超期);图书=ISBN/书名/作者/出版社/分类/馆藏总数/可借库存;借阅=借出(扣库存)→ 到期 → 归还(回补库存)/ 续借(延期);权限=读者只能操作自己的借阅、管理员才能增删改图书。

### Step 2 · 功能清单 + 砍范围(MoSCoW)

```
基于上面的需求,列一份功能清单,并按优先级标注:
- 必做(闭环缺了就不完整):登录注册、图书查询(关键字+分类)、借书、还书、续借、我的借阅、图书增删改、读者列表、借阅记录/超期统计
- 选做(有时间再做):罚款计算、图书详情页浏览量、导出报表
- 不做(这次明确不碰):在线支付、图书封面图上传、消息推送
列成表。
【边界】只列清单不写代码;不确定优先级的先标问号问我。
```

> ⭐ **砍范围最重要**。弱队最容易"什么都想做、什么都做不完"。先锁"必做闭环",选做量力而行。C/S 项目尤其别一上来就想做罚款/封面图,先把"借→还→续借"这条主链跑通。

### Step 3 · 窗口清单 + 分工映射

基于必做功能,列出需要的 Swing 窗口,并把每个功能映射到:服务端哪个命令字(LOGIN/REGISTER/BOOK_QUERY/CATEGORY_LIST/BORROW/RETURN/RENEW/MY_BORROW/BOOK_ADD/BOOK_UPDATE/BOOK_DELETE/READER_LIST/BORROW_RECORD)、客户端哪个窗口。做成"功能→命令字→窗口"对照表。
【边界】只做映射表,不写代码。

> 这张表让分工落地:服务端甲乙照命令组认领,客户端照窗口清单认领。抄进分工看板。典型窗口:登录窗、注册对话框、读者主窗(图书查询/我的借阅两个选项卡)、图书详情+借书对话框、管理员主窗(图书管理/读者管理/借阅记录三个面板)、图书新增编辑对话框、服务端监控窗口。

## Step 4 · 写 Agent PRD + 常驻护栏,存进 `.trae/rules`

把以上确认的内容整理成项目 Agent PRD,写进 .trae/rules,包含这几节:
① 项目概述与角色(读者/管理员)② 核心功能(标必做/选做/不做)③ 技术选型(Java 11 + Swing + Socket 自定义文本协议 + JDBC + MySQL,三段式:common 共享 / server 服务端 / client 客户端,不引第三方框架,JSON 手写)④ 数据模型(先占位,环节2 填)⑤ 通信协议规范(统一响应 OK|数据 / ERR|原因、请求一行一条 命令|参数、列表用手写 Json 编码单行、鉴权靠 userId+role;详细契约环节3 填)⑥ 编码规范(表 t_ 前缀、字段下划线、类命名前缀隔离)⑦ 界面风格。
另外,在最前面加一节「全局铁律」,内容就是本课发的 7 条护栏(只做当前这步/不新建未提及的文件表命令依赖/不顺手重构/不动范围外文件/不自作主张加功能/不确定先问/每次报告改了哪些文件)。
【边界】只写 .trae/rules 这一个文件,不要生成任何业务代码。

> **这一步同时把 §通用护栏的常驻规则写进了 `.trae/rules`**——从此每次生成 AI 都受约束。

# ✅ PRD 自查表(缺哪补哪)

- [ ] 角色和权限写清(谁能借还、谁能管图书、读者只能碰自己的借阅)
- [ ] 功能标了必做/选做/不做(范围可控)
- [ ] 每个功能有"命令字 + 客户端窗口"归属(对得上分工)
- [ ] 技术选型明确、统一响应/请求格式/鉴权方式定了
- [ ] 编码规范(命名/前缀隔离)写了
- [ ] **`.trae/rules` 里已包含 7 条全局护栏**
- [ ] 数据模型、协议留了占位(环节 2/3 填)

# 常见踩坑 + 纠错提示词

| 现象 | 纠错提示词 |
| --- | --- |
| AI 在写 PRD 时顺手把代码也生成了 | `你超范围了:这一步只写 .trae/rules,把生成的代码撤掉。` |
| 需求贪多,功能清单一大堆 | `按 MoSCoW 重排,只保留借/还/续借这条主链 + 图书管理做闭环,其余移到选做。` |
| PRD 写得空,没法指导开发 | `每个功能补一句"谁操作、发什么命令、返回什么结果",别写空话。` |

# 环节 2 · 数据库设计(数据字典先行) 🔒

> **本环节主责**:组长 + 服务端甲乙 一起设计;文档维护;测试评审。 **为什么放这么前**:数据库是全组的地基。表定不细,AI 写服务端时字段对不上、写客户端时不知道显示啥,就会**边写边改表**,一改全组返工。**这一步的产出锁定后,不许再随便改表**——要改走"变更评审"。 **铁律**:动任何业务代码前,先完成本环节并把数据字典写进 `.trae/rules`。库名 `library`,字符集 `utf8mb4`,MySQL `127.0.0.1:3306` `root/12345678`。

## 多步提示词(依次喂给 AI)

### Step 1 · 从 PRD 抽实体清单

```
这是我们的项目 PRD(见 .trae/rules)。请你像数据库设计师一样,从中抽出需要建的数据库表(实体)清单。
每个实体给:表名(t_ 前缀)、一句话作用、和其它表的关系(谁属于谁/谁引用谁)。
先只列清单和关系,不要写字段。列完停下等我确认。
```

> **讲师看点**:让 AI 先给"实体地图",全组对着确认——图书馆至少要有:用户、图书分类、图书、借阅记录(罚款可选)。确认了再往下,别让 AI 一口气把表和字段全喷出来(容易糊)。

### Step 2 · 逐表输出数据字典

```
按下面的字段模板,给「t_book 图书表」输出完整数据字典,一行一个字段:
| 字段名 | 类型(长度) | 是否可空 | 默认值 | 键/约束 | 说明 |
要求:
- 主键 id 用 INT 自增;
- total 馆藏总数、stock 当前可借库存都用 INT;
- 分类 category_id 标出引用 t_category;
- 字符串字段(isbn/title/author/publisher)给合理长度,书名可较长。
先只做 t_book 这一张,做完停下我看。
```

> 逐表来(t_user → t_category → t_book → t_borrow → t_fine),**一次一张**,每张确认再下一张。这样字段能盯得住。

### Step 3 · 补齐"每张表必备"

```
回看你给的全部数据字典,按这份清单逐条自检并补齐:
① 每张表有没有 id 主键?需要 create_time 的表(如 t_user)有没有?
② 库存类字段(total/stock)是不是 INT?罚款金额是不是 DECIMAL?
③ 状态类字段是不是用固定字面量 + 说明写清?(如借阅
status='borrowed'/'returned';return_time 为 NULL 表示未还)
④ 借阅记录表有没有 user_id、book_id(查"我的借阅"和联表带书名/读者名靠它们)?
⑤ 用户密码字段长度够不够(VARCHAR(100))?
把补齐后的完整数据字典重新给我。
```

> ⭐ **这一步最防坑**。图书馆最容易漏的是**借阅状态的两套表达**:一是 `t_borrow.status`('borrowed'/'returned'),二是 `t_borrow.return_time`(NULL=未还)——两者要联动一致(还书时同时置 status='returned' 且 return_time=NOW())。让 AI 自检补齐,而不是等开发到借还模块才发现表不对。

## Step 4 · 关系 + 约束 + 索引

```
基于确认的数据字典,补三件事:
① 列出所有外键关系(book.category_id→category.id、borrow.user_id→user.id、
borrow.book_id→book.id、fine.borrow_id→borrow.id);
② 需要唯一约束的字段(user.username 唯一、category.name 唯一);
③ 该加索引的列(所有外键列 + 常筛选列 borrow.status)。
给我一张"关系+约束+索引"汇总表。
```

> ⚠️ **本 C/S 答案版用了物理外键**(`CONSTRAINT ... FOREIGN KEY`)。这是与本训练营 B/S 参考案例(教学**不建**物理外键、靠应用层校验)不同的一处取舍——C/S 单机演示、数据量小,用物理外键能让"删有借阅记录的书"这类操作直接被数据库拦住,便于教学演示外键约束的效果。代价见环节 5 踩坑(删书报外键约束错)。各组二选一即可,**跟着自己答案版走、写进契约**。

## Step 5 · 生成 ER 图

```
用 Mermaid 的 erDiagram 语法,把 t_user / t_category / t_book / t_borrow / t_fine 五
张表和它们的关系画成 ER 图,标出主外键和 1:N 关系。只输出 Mermaid 代码块。
```

> 文档岗把这段 Mermaid 存进数据字典文档,答辩 PPT 直接用。

## Step 6 · 生成建库 SQL + 种子

```
按最终确认的数据字典,生成 MySQL 建库脚本:
- 库名 library(utf8mb4);
- 5 张表的 CREATE TABLE(带字段注释、主键、外键、唯一约束);
- 种子数据:管理员 admin/123456 + 读者 zhangsan/123456、lisi/123456;3 个分类;8 本图书
(覆盖三个分类,含一本 stock=1 便于演示并发防超借)。
只输出可直接执行的 SQL。
```

# 五表完整数据字典(已对照答案版 `sql/init.sql` 核对一致,作为验收基准)

## t_user 用户表(读者 + 管理员)

| 字段名 | 类型(长度) | 可空 | 默认 | 键/约束 | 说明 |
|---|---|---|---|---|---|
| id | INT | 否 | 自增 | PK | 主键 |
| username | VARCHAR(50) | 否 | — | UNIQUE | 登录名 |
| password | VARCHAR(100) | 否 | — | | 密码(教学项目明文,生产应加密) |
| real_name | VARCHAR(50) | 否 | — | | 真实姓名 |
| role | VARCHAR(10) | 否 | 'reader' | | 角色:`reader` 读者 / `admin` 管理员 |
| max_borrow | INT | 否 | 5 | | 可借上限 |
| create_time | DATETIME | 否 | CURRENT_TIMESTAMP | | 注册时间 |

## t_category 图书分类表

| 字段名 | 类型(长度) | 可空 | 默认 | 键/约束 | 说明 |
|---|---|---|---|---|---|
| id | INT | 否 | 自增 | PK | 主键 |
| name | VARCHAR(50) | 否 | — | UNIQUE | 分类名 |

## t_book 图书表

| 字段名 | 类型(长度) | 可空 | 默认 | 键/约束 | 说明 |
|---|---|---|---|---|---|
| id | INT | 否 | 自增 | PK | 主键 |
| isbn | VARCHAR(30) | 否 | '' | | ISBN 号 |
| title | VARCHAR(200) | 否 | — | | 书名 |
| author | VARCHAR(100) | 否 | '' | | 作者 |
| publisher | VARCHAR(100) | 否 | '' | | 出版社 |
| category_id | INT | 是 | NULL | FK→t_category.id | 分类外键 |
| total | INT | 否 | 0 | | 馆藏总数 |
| stock | INT | 否 | 0 | | 当前可借库存(借出减 1、归还加 1) |

> **派生字段单独标:**`categoryName`(分类名)**不是 t_book 的列**,是 `BOOK_QUERY` 查询时 `LEFT JOIN t_category` 带出来的展示字段。数据字典里标注"非表列,JOIN 带出",客户端表格要显示它。

## t_borrow 借阅记录表

| 字段名 | 类型(长度) | 可空 | 默认 | 键/约束 | 说明 |
|---|---|---|---|---|---|
| id | INT | 否 | 自增 | PK | 主键 |
| user_id | INT | 否 | — | FK→t_user.id | 借书人 |
| book_id | INT | 否 | — | FK→t_book.id | 所借图书 |
| borrow_time | DATETIME | 否 | — | | 借出时间 |
| due_time | DATETIME | 否 | — | | 应还时间(借出 + 30 天;续借再 + 30 天) |
| return_time | DATETIME | 是 | NULL | | 实际归还时间,**NULL 表示未还** |
| status | VARCHAR(10) | 否 | 'borrowed' | | `borrowed` 在借 / `returned` 已还 |

> **派生字段单独标**:`bookTitle`(书名,JOIN `t_book`)、`readerName`(读者姓名,JOIN `t_user`,仅管理员的借阅记录列表用)**都不是表列**,是查询时联表带出的展示字段。

## t_fine 罚款表(可选,超期归还时生成)

| 字段名 | 类型(长度) | 可空 | 默认 | 键/约束 | 说明 |
|---|---|---|---|---|---|
| id | INT | 否 | 自增 | PK | 主键 |
| borrow_id | INT | 否 | — | FK→t_borrow.id | 关联的借阅记录 |
| amount | DECIMAL(10,2) | 否 | 0.00 | | 罚款金额 |
| paid | TINYINT | 否 | 0 | | 0 未缴 / 1 已缴 |

> ⚠️ **答案版建了 t_fine 表但未实现罚款业务逻辑**(属"选做"),协议里也没有对应命令。要做罚款的组:超期归还时按超期天数生成 t_fine 记录,并加一条如 `FINE_LIST` 的命令——记得同步进环节 3 协议契约。

# 建库 SQL(节选,与答案版 `sql/init.sql` 一致,作为验收基准)

```
CREATE DATABASE IF NOT EXISTS library DEFAULT CHARSET utf8mb4 COLLATE
utf8mb4_general_ci;
USE library;

-- 重建顺序:先删子表(带外键的)
DROP TABLE IF EXISTS t_fine;
DROP TABLE IF EXISTS t_borrow;
DROP TABLE IF EXISTS t_book;
DROP TABLE IF EXISTS t_category;
DROP TABLE IF EXISTS t_user;

CREATE TABLE t_user (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  username    VARCHAR(50)  NOT NULL UNIQUE COMMENT '登录名',
  password    VARCHAR(100) NOT NULL        COMMENT '密码(教学明文, 生产应加密)',
  real_name   VARCHAR(50)  NOT NULL        COMMENT '真实姓名',
  role        VARCHAR(10)  NOT NULL DEFAULT 'reader' COMMENT 'reader / admin',
```

```
  max_borrow  INT          NOT NULL DEFAULT 5 COMMENT '可借上限',
  create_time DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='用户表';

CREATE TABLE t_category (
  id   INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(50) NOT NULL UNIQUE COMMENT '分类名'
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='图书分类表';

CREATE TABLE t_book (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  isbn        VARCHAR(30)  NOT NULL DEFAULT '' COMMENT 'ISBN 号',
  title       VARCHAR(200) NOT NULL           COMMENT '书名',
  author      VARCHAR(100) NOT NULL DEFAULT '' COMMENT '作者',
  publisher   VARCHAR(100) NOT NULL DEFAULT '' COMMENT '出版社',
  category_id INT          NULL                COMMENT '分类外键',
  total       INT          NOT NULL DEFAULT 0  COMMENT '馆藏总数',
  stock       INT          NOT NULL DEFAULT 0  COMMENT '当前可借库存',
  CONSTRAINT fk_book_category FOREIGN KEY (category_id) REFERENCES t_category(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='图书表';

CREATE TABLE t_borrow (
  id          INT PRIMARY KEY AUTO_INCREMENT,
  user_id     INT      NOT NULL COMMENT '借书人',
  book_id     INT      NOT NULL COMMENT '所借图书',
  borrow_time DATETIME NOT NULL COMMENT '借出时间',
  due_time    DATETIME NOT NULL COMMENT '应还时间',
  return_time DATETIME NULL     COMMENT '实际归还时间, NULL 表示未还',
  status      VARCHAR(10) NOT NULL DEFAULT 'borrowed' COMMENT 'borrowed /
returned',
  CONSTRAINT fk_borrow_user FOREIGN KEY (user_id) REFERENCES t_user(id),
  CONSTRAINT fk_borrow_book FOREIGN KEY (book_id) REFERENCES t_book(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='借阅记录表';

CREATE TABLE t_fine (
  id        INT PRIMARY KEY AUTO_INCREMENT,
  borrow_id INT           NOT NULL COMMENT '关联的借阅记录',
  amount    DECIMAL(10,2) NOT NULL DEFAULT 0.00 COMMENT '罚款金额',
  paid      TINYINT       NOT NULL DEFAULT 0 COMMENT '0 未缴 / 1 已缴',
  CONSTRAINT fk_fine_borrow FOREIGN KEY (borrow_id) REFERENCES t_borrow(id)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='罚款表';

-- 种子:管理员 admin + 读者 zhangsan/lisi(均 123456)
INSERT INTO t_user (username, password, real_name, role, max_borrow) VALUES
  ('admin',    '123456', '系统管理员', 'admin',  99),
  ('zhangsan', '123456', '张三',       'reader', 5),
  ('lisi',     '123456', '李四',       'reader', 3);

INSERT INTO t_category (name) VALUES ('计算机'), ('文学'), ('历史');

-- 8 本图书(含《万历十五年》stock=1,用于演示并发防超借)
```

```
INSERT INTO t_book (isbn, title, author, publisher, category_id, total, stock)
VALUES
  ('9787111213826', '深入理解计算机系统', 'Randal E. Bryant', '机械工业出版社', 1, 5,
5),
  ('9787115428028', 'Java核心技术卷I',    'Cay S. Horstmann', '人民邮电出版社', 1, 4,
4),
  ('9787121362217', '算法导论',           'Thomas H. Cormen', '电子工业出版社', 1,
3, 3),
  ('9787020024759', '红楼梦',             '曹雪芹',           '人民文学出版社', 2, 6,
6),
  ('9787532736553', '百年孤独',           '加西亚·马尔克斯', '上海译文出版社', 2, 2,
2),
  ('9787506365437', '活着',               '余华',             '作家出版社',     2,
3, 3),
  ('9787101003048', '史记',               '司马迁',           '中华书局',       3,
2, 2),
  ('9787108009821', '万历十五年',         '黄仁宇',           '生活·读书·新知三联书店',
3, 1, 1);
```

> ⚠️ **种子只建了图书,没有预置借阅记录**(t_borrow 初始为空)。演示"在借/已还/超期"三种借阅状态时,现场用读者账号借几本、还一本即可产生;若答辩要一开场就有历史记录,可自行补几条 t_borrow 种子(注意同步扣减对应图书 stock,保证 stock 与在借数一致)。

## 图书库存 × 借阅状态 联动(借还模块最易踩坑,建库前就定清)

| 动作 | t_book.stock | t_borrow.status | t_borrow.return_time |
|---|---|---|---|
| 借书 `BORROW` | `stock - 1`(条件 `WHERE stock>0`) | 新建记录 = `borrowed` | NULL(未还) |
| 达借阅上限拦截 | 不变(事务回滚) | 不新建 | — |
| 还书 `RETURN` | `stock + 1` | `borrowed → returned`(条件 `WHERE status='borrowed'`) | 置为 `NOW()` |
| 续借 `RENEW` | 不变 | 保持 `borrowed` | 保持 NULL(仅 `due_time + 30 天`) |
| 重复还/续已还记录 | 不变 | 拦截(影响 0 行 → 报错) | — |

> - **两条不变量**,建库和写业务时都要守住:① `stock` 永远 = `total - 该书当前在借数`(borrowed 且未还的记录数);② `status='returned'` 必然 `return_time IS NOT NULL`,`status='borrowed'` 必然 `return_time IS NULL`。
> - **借阅上限**:借书前先查该读者 `max_borrow` 与"未还数量"(`status='borrowed'` 计数),达上限则拒绝(admin=99、zhangsan=5、lisi=3,新注册读者默认 5)。
> - **默认借期 30 天,续借再加 30 天**(答案版 `BorrowService.BORROW_DAYS=30`、`RENEW_DAYS=30`);要改借期改这两个常量。
> - **超期判定**:`status='borrowed'` 且 `due_time < NOW()` 即超期;管理员 `BORROW_RECORD|1` 只筛超期记录。

## ✅ 数据库评审 checklist(锁 schema 前,测试 + 组长逐条打勾)

- [ ] 命名统一:表 `t_` 前缀、字段全下划线;服务端读结果集时字段名映射成驼峰(如 `real_name` → `realName`、`category_id` → `categoryId`)
- [ ] 每张表都有 `id`(INT 自增)主键;需要时间的表有 `create_time`
- [ ] 库存/上限用 `INT`,罚款金额用 `DECIMAL(10,2)`,没有 float/double
- [ ] 借阅状态用固定字面量 `borrowed` / `returned` + 注释写清,且与 `return_time` 是否为 NULL 联动一致
- [ ] 唯一约束:`user.username`、`category.name`
- [ ] **越权隔离依据**:借阅记录有 `user_id`(查"我的借阅"、校验是本人靠它)
- [ ] 外键关系全部明确(book→category、borrow→user/book、fine→borrow);**本答案版建了物理外键**(与 B/S 案例取舍不同,写进契约)
- [ ] 密码字段 `VARCHAR(100)`
- [ ] **库存不变量**:`stock ≤ total`,借减还加,不超借不透支(靠环节 3 的原子 UPDATE 保证)
- [ ] 种子覆盖:管理员 + 2 读者 + 3 分类 + 8 图书(含 stock=1 一本用于并发演示)

## 常见踩坑 + 纠错提示词

| 现象 | 纠错提示词 |
| --- | --- |
| AI 把 stock/total 设成 VARCHAR 或漏了 stock | `total 馆藏、stock 可借库存都用 INT;stock 是借出减1/归还加1 的动态列,不能少。只改 t_book。` |
| 借阅只用 status 不留 return_time(或反之) | `t_borrow 要同时有 status(borrowed/returned)和 return_time(NULL=未还),还书时两者一起改。只改 t_borrow 定义。` |
| 分类名直接冗余进 t_book | `categoryName 不建进 t_book,它是查询时 LEFT JOIN t_category 带出的派生字段。把 t_book 里的分类名列删掉,只留 category_id。` |

---

# 环节 3 · 通信协议设计(契约先行) 🔒

> **本环节主责**:组长牵头,服务端甲乙定各自命令组协议,客户端确认够用,测试评审,文档维护协议文档。 **为什么重要**:数据库锁的是"数据边界",协议锁的是"协作边界"。**协议一定,客户端和服务端 2 人就能各写各的、互不阻塞**;联调时按协议对,对不上就是谁没守协议。这是横切不翻车的命脉。 **铁律**:协议锁定前,客户端不写窗口、服务端不写命令分支。协议常量集中写在 `common/Protocol.java`,是唯一事实源。

## 全局约定(先定这 5 条,后面每条命令都遵守)

1. **统一响应**:成功 `OK|数据`,失败 `ERR|原因`。客户端按响应**第一个 `|` 之前**的标志判成败:等于 `OK` 取后面 payload 用;否则把后面的原因弹提示。(答案版 `client/net/Api.send()` 就这么判。)
2. **请求一行一条**:请求格式固定 `命令|参数1|参数2...`,一行一条;响应也是一行一条。收发两端都用 `readLine()` 按行读、每条以 `\n` 结尾——**"一行一条 + readLine" 天然规避 TCP 粘包/半包**,不要自己按字节拼。
3. **列表数据用手写 Json 编码成单行**:图书列表、借阅列表等多行数据,由 `common/util/Json.java` 编码成**不含裸 `|` 和换行**的单行字符串放进 payload。Json 内部把 `\ | = 换行` 等全部转义(用不可见字符 FS/RS 分隔字段/记录),所以外层协议按 `|` 拆分绝不会与数据内容冲突。**列表一律走 Json.encode,禁止手工字符串拼接**。
4. **鉴权靠 userId + role**:协议无 token。登录成功后客户端 `Session` 保存 `userId/role`;需要身份的命令(如 `BORROW`/`MY_BORROW`)把 `userId` 作为参数带上;**管理类命令(BOOK_ADD/UPDATE/DELETE、READER_LIST、BORROW_RECORD)语义上仅管理员可用**——答案版目前靠客户端"按 role 分流窗口"实现(读者窗口根本没有这些入口),服务端未做角色校验。**这正是环节 5 越权测试要补的硬伤**:客户端藏了入口不等于安全,服务端应在命令分支入口按 role 拦截(见环节 5)。
5. **分工前缀(直接对应服务端 2 人)**:
   - 服务端甲:`LOGIN` `REGISTER` `BOOK_QUERY` `CATEGORY_LIST`
   - 服务端乙:`BORROW` `RETURN` `RENEW` `MY_BORROW` `BOOK_ADD` `BOOK_UPDATE` `BOOK_DELETE` `READER_LIST` `BORROW_RECORD`

# 多步提示词

## Step 1 · 按命令组列协议清单

```
基于 .trae/rules 的功能和数据模型,按模块列出所有通信命令清单(先只列:命令字 / 请求参数概要 / 一句话作用,不写完整字段)。
分两组:①登录注册 + 图书查询 + 分类(服务端甲)②借还续借 + 我的借阅 + 图书管理 + 读者列表 + 借阅记录(服务端乙)。列成表,列完停下。
```

> 全组对着清单确认"够不够、有没有重"。服务端甲乙照 §分工前缀 认领。

## Step 2 · 逐命令写契约

```
按下面模板,给「借还模块」的每个命令写完整契约:
### 命令字 CMD
- 作用:
- 权限:(公开 / 登录读者 / 仅本人 / 仅管理员)
- 请求:CMD|参数1|参数2 (逐个参数:名/含义/是否必填)
- 成功响应:OK|... (逐字段说明;列表说明 Json 里每行有哪些 key,对齐数据字典)
- 失败响应:ERR|原因 (可能的原因逐条列)
先只做借还模块,做完停下。
```

> 一个命令组一个命令组来。成功响应里的字段必须**对齐环节 2 的数据字典**(驼峰),客户端才知道窗口能拿到啥。

## Step 3 · 客户端拿契约走查每个窗口

(客户端做)我有这些窗口:登录、注册、图书查询表格、图书详情+借书、我的借阅、图书管理表格、读者管理、借阅记录/超期。
请你对照上面的命令契约,逐个窗口检查:这个窗口要显示/提交的数据,契约里的命令都能提供吗?
缺什么(比如图书查询表格要显示分类名,但 BOOK_QUERY 的返回没有 categoryName)就列出来。

> **客户端确认"够不够用"是关键一步**。典型缺口:图书表格要显示"分类名",但 t_book 只有 `categoryId`;借阅记录要显示"书名、读者名",但 t_borrow 只有 `book_id`/`user_id`。**走查发现后,就在契约的响应里加上 `categoryName`/`bookTitle`/`readerName`**(服务端联表带出)——这就是下面契约里这些派生字段的来源,不是凭空来的。

## Step 4 · 测试评审(挑刺)

(测试做)请以"找漏洞"的视角评审这套命令契约:
① 越权:读者能不能发 BOOK_DELETE / READER_LIST / BORROW_RECORD?服务端拦不拦?查"我的借阅"怎么保证只看到自己的?
② 边界:借一本 stock=0 的书怎么办?还一条已经还过的记录怎么办?续借一条已还记录呢?达借阅上限再借呢?
③ 缺失:超期要不要罚款命令?图书详情要不要单独命令?
把问题按"命令/风险/建议"列出来。

## Step 5 · 锁定协议文档

把定稿的全部命令契约整理成一份通信协议文档,追加进 .trae/rules,并把命令字常量集中定义到 common/Protocol.java。开发阶段两端都以它为准,不许私自改命令字和参数顺序;要改必须全组过一遍。

# 完整协议契约(13 条,已用答案版工程 `cs-swing-library-demo` 跑通验证,作为验收基准)

> 通用:请求 `命令|参数...`;成功 `OK|数据`,失败 `ERR|原因`。列表型 payload 由手写 Json 编码成单行。

```
### LOGIN                                        [服务端甲]
- 作用:登录,按角色分流读者/管理员窗口
- 权限:公开
- 请求:LOGIN|username|password
- 成功:OK|role|realName|userId          // role=reader/admin;三段直接用 | 拼(非列表)
- 失败:ERR|账号或密码错误

### REGISTER                                     [服务端甲]
- 作用:读者注册,注册即得 userId(默认可借上限 5、角色 reader)
- 权限:公开
- 请求:REGISTER|username|password|realName
- 成功:OK|userId
- 失败:ERR|用户名已存在  /  ERR|用户名和密码不能为空
```

```
### BOOK_QUERY                              [服务端甲]
- 作用:图书查询(关键字模糊 + 分类过滤)
- 权限:登录(读者/管理员皆可)
- 请求:BOOK_QUERY|关键字|分类id              // 关键字可空;分类id=0 表示全部分类
- 成功:OK|图书列表(Json)
        每行 key:id, isbn, title, author, publisher, categoryId, categoryName,
total, stock
        // categoryName 为 LEFT JOIN t_category 带出的派生字段
- 失败:ERR|原因

### CATEGORY_LIST                           [服务端甲]
- 作用:全部分类(供客户端分类下拉)
- 权限:登录
- 请求:CATEGORY_LIST
- 成功:OK|分类列表(Json)  每行 key:id, name
- 失败:ERR|原因

### BORROW                                  [服务端乙]
- 作用:借书(事务:查上限 → 扣库存 → 写借阅记录)
- 权限:登录读者(userId 为本人)
- 请求:BORROW|userId|bookId
- 成功:OK|应还日期                          // 形如 2026-07-08 15:30:00(借出时刻 + 30
天)
- 失败:ERR|库存不足,借阅失败  /  ERR|已达借阅上限(N 本),请先还书
- 实现约束(写进契约):扣库存用 UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0,
        按影响行数判成败(0 行=无库存,回滚返 ERR);扣库存 + 插记录同一事务

### RETURN                                  [服务端乙]
- 作用:还书(事务:借阅记录置 returned + 库存 +1)
- 权限:登录读者(本人借阅)
- 请求:RETURN|borrowId
- 成功:OK
- 失败:ERR|该记录不存在或已归还            // UPDATE ... WHERE status='borrowed' 影响
0 行即防重复还

### RENEW                                   [服务端乙]
- 作用:续借(due_time 再延 30 天,仅 borrowed 可续)
- 权限:登录读者(本人借阅)
- 请求:RENEW|borrowId
- 成功:OK|新应还日期
- 失败:ERR|该记录不存在或已归还,无法续借

### MY_BORROW                               [服务端乙]
- 作用:我的借阅(某读者的全部借阅记录)
- 权限:登录读者(userId 为本人)
- 请求:MY_BORROW|userId
- 成功:OK|借阅列表(Json)
        每行 key:id, bookId, bookTitle, borrowTime, dueTime, returnTime, status
        // bookTitle 为 JOIN t_book 带出;不含 readerName
- 失败:ERR|原因

### BOOK_ADD                                [服务端乙]
```

```
- 作用:新增图书
- 权限:仅管理员
- 请求:BOOK_ADD|图书map(Json 单条编码)      //
key:isbn,title,author,publisher,categoryId,total,stock
- 成功:OK|newId
- 失败:ERR|书名不能为空  /  ERR|库存不能大于馆藏总数  /  ERR|馆藏数 / 库存不能为负

### BOOK_UPDATE                          [服务端乙]
- 作用:编辑图书
- 权限:仅管理员
- 请求:BOOK_UPDATE|图书map(Json 单条,含 id)
- 成功:OK
- 失败:ERR|缺少图书 id  /  ERR|书名不能为空  /  ERR|库存不能大于馆藏总数

### BOOK_DELETE                          [服务端乙]
- 作用:删除图书
- 权限:仅管理员
- 请求:BOOK_DELETE|bookId
- 成功:OK
- 失败:ERR|服务端异常: ...                  // 该书已有借阅记录时,被物理外键
fk_borrow_book 拦下(见环节 5 踩坑)

### READER_LIST                          [服务端乙]
- 作用:读者列表(管理员查读者)
- 权限:仅管理员
- 请求:READER_LIST
- 成功:OK|读者列表(Json)  每行 key:id, username, realName, role, maxBorrow
- 失败:ERR|原因

### BORROW_RECORD                        [服务端乙]
- 作用:全部借阅记录 / 超期统计(管理员)
- 权限:仅管理员
- 请求:BORROW_RECORD|onlyOverdue          // 0=全部记录;1=只看超期(borrowed 且
due_time<now)
- 成功:OK|借阅记录列表(Json)
        每行 key:id, bookId, bookTitle, readerName, borrowTime, dueTime,
returnTime, status
        // 比 MY_BORROW 多 readerName(JOIN t_user 带出)
- 失败:ERR|原因
```

# ✅ 协议评审 checklist(锁定前)

- [ ] 每条命令标了**权限**(公开/登录读者/仅本人/仅管理员)
- [ ] 列表型响应都走 Json 编码(不手工拼 `|` 列表)
- [ ] 成功响应字段名和数据字典一致(驼峰),客户端要的派生字段都在(`categoryName` / `bookTitle` / `readerName`)
- [ ] 写操作(借/还/续、图书增删改)的成功/失败响应都写清了
- [ ] 借书命令在实现约束里写了**事务 + 原子扣库存防超借**
- [ ] 失败原因文案约定清楚(客户端直接弹给用户)

- [ ] 命令字不跨服务端 2 人的分界,且集中定义在 `Protocol.java`
- [ ] **越权隐患已登记**:管理类命令服务端未校验 role(环节 5 补)

## 服务端 2 人据契约认领(直接抄进分工看板)

| 命令组 | Owner | 交接线(不碰对方) |
| --- | --- | --- |
| `LOGIN` `REGISTER` `BOOK_QUERY` `CATEGORY_LIST` | 服务端甲 | 只建/改 User/Category 相关类 + `BookDao.query`;`ClientHandler` 只加这 4 个 case |
| `BORROW` `RETURN` `RENEW` `MY_BORROW` 图书增删改 `READER_LIST` `BORROW_RECORD` | 服务端乙 | 只建/改 Borrow 相关类 + `BookDao.insert/update/delete`;`ClientHandler` 只加这 9 个 case |
| 共享 `UserDao` / `BookDao` | 甲写、乙只读调用 | 乙的 `READER_LIST` 读甲的 `UserDao.listReaders()`;`BookDao` 按方法切,谁写的方法谁维护、不改对方签名 |

## 常见踩坑 + 纠错提示词

| 现象 | 纠错提示词 |
| --- | --- |
| 图书列表客户端解析乱掉(书名含标点被误拆) | `列表数据必须走 Json.encode/decode(它把 \| = 换行都转义),不要手工用 \| 拼列表。改回用 Json,别动别的命令。` |
| 服务端甲乙把同一命令字都写了(重复) | `按契约的分工前缀,BOOK_QUERY 归甲、BOOK_ADD 归乙。撤掉你写的越界那条 case。` |
| 客户端把 BORROW 的 userId/bookId 传反 | `按 Protocol 常量注释:BORROW\|userId\|bookId,userId 在前。只改 Api.borrow 的参数顺序。` |
| 响应字段名两端不一致(real_name vs realName) | `统一按契约:Json 里用驼峰 realName。只改这个字段名,别的不动。` |

---

# 环节 4 · 分模块开发

> **本环节主责**:服务端甲乙、客户端各主责自己那块;组长每半天集成;测试单命令边测。 **定位**:协议锁定后,**服务端甲、服务端乙、客户端三条线并行**。每个模块都是"照契约 + `.trae/rules` 的规则,加一块";客户端用"假数据/桩先行"不被服务端卡住。

## 多步提示词

### 服务端 · 加一个命令组(以借还模块为例,服务端乙)

```
基于现有骨架和 .trae/rules 的数据模型与协议契约,加借还模块(只做
BORROW/RETURN/RENEW/MY_BORROW):
- BorrowDao:findByUser(userId,联 t_book 带 bookTitle)、findAll(onlyOverdue,联
t_book/t_user 带 bookTitle/readerName)
- BorrowService:
    borrow(userId,bookId):setAutoCommit(false) 事务 → 查 max_borrow 与未还数(达上限
抛异常回滚)→ UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0 判影响行数(0 行
抛"库存不足")→ INSERT t_borrow(borrowed,due=now+30天)→ commit
    returnBook(borrowId):事务 → UPDATE t_borrow SET
status='returned',return_time=NOW() WHERE id=? AND status='borrowed' 判影响行数 →
UPDATE t_book SET stock=stock+1
    renew(borrowId):UPDATE due_time=DATE_ADD(due_time,INTERVAL 30 DAY) WHERE id=?
AND status='borrowed'
    myBorrow(userId)、allRecords(onlyOverdue)
- 在 ClientHandler 里补 BORROW/RETURN/RENEW/MY_BORROW 这 4 个 case,严格按契约的请求/
响应格式(列表走 Json.encode)
【边界】只做借还这 4 个命令,不新建登录/图书查询相关文件,不改公共基建
(Protocol/Json/DBUtil/ClientHandler 骨架),不加契约没有的命令;做完列出改了哪些文件。
```

> 讲师审查点:让 AI 逐行解释 `borrow()` 的事务——`setAutoCommit(false)` 关自动提交、原子 `UPDATE ... WHERE stock>0` 按影响行数判成败、任一步失败 `rollback()`。这是本项目答辩重点,也是防超借的命脉。

## 客户端 · 桩/假数据先行(客户端,不等服务端)

```
按契约写"图书查询面板"(BookQueryPanel),服务端还没好,先用假数据渲染:
- 顶部搜索栏(关键字输入框 + 分类下拉 + 查询按钮)+ 中间 JTable(列:书名/作者/出版社/分类名
categoryName/可借库存stock)+ 双击某行弹图书详情借书对话框
- 先在面板里放 3 条假的 Book 数据(字段照契约 BOOK_QUERY 返回的 key)把界面撑出来
- 网络请求要走后台线程(SwingWorker),别在事件线程里直接读 socket(会卡界面)
【边界】只写这个面板 + 假数据,不接真命令、不动别的窗口。
```

(服务端好了再联调换真调用)

```
把图书查询面板的假数据换成真调用:调 Api.queryBooks(keyword,categoryId) 拿列表;分类下拉调
Api.categories()。
【边界】只改这个面板的取数逻辑,别的不动。
```

> **桩先行 = 客户端不被服务端卡住**,协议就是两边的对齐点。这是横切能并行的关键。客户端全程只碰 `client/ui/**` 和 `Api.java`。

## 集成节奏(组长,每半天末)

合并甲/乙的命令分支(手工把两人的 case 合进同一个 `ClientHandler` switch)+ 客户端窗口 → 起服务端(先建库)→ 起客户端 → 冒烟点一遍主流程 → 记问题。**别攒到最后一天合**。

# ✅ 环节 4 验收 checklist

- [ ] 服务端甲乙各只碰自己命令组的 case + 前缀文件(文件清单核对,没串门)

- [ ] `ClientHandler` 的两人 case 已合并进同一个 switch,无重复/无覆盖
- [ ] 客户端按契约解析,`categoryName` / `bookTitle` 等派生字段显示出来了
- [ ] 网络请求走 SwingWorker,界面不卡死
- [ ] 每半天集成过一次(合并→起服务端+客户端→冒烟)
- [ ] AI 没越界改别人命令分支/公共基建(看它每次的改动清单)

## 常见踩坑 + 纠错提示词

| 现象 | 纠错提示词 |
| --- | --- |
| AI 写借书顺手把登录/分页/缓存都加了 | `你超范围了,只做借还 4 个命令,把多加的登录/缓存撤掉。` |
| 服务端甲乙都改了公共 `Protocol` / `ClientHandler` 骨架 | `公共基建只有组长维护,把你对 Protocol 的改动还原,有需求提给组长。你只加自己命令组的 case。` |
| 客户端在事件线程直接读 socket,点按钮界面冻住 | `把这个网络请求挪到 SwingWorker 的 doInBackground 里,拿到结果再在 done 里刷新表格。只改这个面板。` |
| 客户端干等服务端 | `先按契约用假数据把面板做出来,服务端好了再换真调用。` |

---

# 环节 5 · 联调 + 测试

> **本环节主责**:测试岗主导;服务端/客户端修 bug;组长盯集成。 **定位**:客户端接真调用时坑集中爆发;测试岗按 `templates/测试用例表.md` 系统测。

## 联调六坑(答案版实测,照着排)

1. **连接失败**(客户端点登录就报网络错误)→ 三查:① 服务端起没起(看服务端监控窗口是否"运行中、监听 9000")② IP/端口填对没(单机 `127.0.0.1:9000`,局域网填服务器电脑 IP)③ 防火墙拦没拦入站 TCP 9000(Windows Defender 常默认拦,首次运行选"允许"或手动加入站规则)。可用 `telnet 服务器IP 9000` 快速验通不通。
2. **粘包 / 半包**(收到的响应少半截或两条粘一起)→ TCP 是字节流、没有消息边界。解法:**约定一行一条**,发送方每条以 `\n` 结尾并 `flush()`,接收方 `readLine()` 按行读——天然分帧。**不要**自己 `read(byte[])` 拼字节。
3. **协议字段对不上**(命令能通但结果错)→ 请求参数顺序/个数写错(如 `BORROW` 把 `userId` / `bookId` 传反)。解法:以 `common/Protocol.java` 常量注释为**唯一契约**,客户端 `Api` 与服务端 `ClientHandler` 对着同一份注释写。
4. **多线程共享状态被写坏**(在线数忽大忽小、日志错乱)→ 每个客户端连接一个 `ClientHandler` 线程,共享的"在线数/日志/监控回调"必须线程安全。答案版用 `AtomicInteger` 计在线数、`volatile Listener` 挂监控窗口;`service` 无状态、每线程各持一份。**别用普通 `int++` 或 `ArrayList`** 让多个连接线程并发改。

5. **JDBC 字符集致中文乱码**(书名/姓名变问号)→ 连接 url 必须 `characterEncoding=UTF-8`(**Java 字符集名**),**不能写** `utf8mb4`(那是 MySQL 服务端字符集名,会报 `Unsupported character encoding`;这是踩过的坑 CROSS-157)。库/表建成 `utf8mb4` 没问题,是 url 参数这里只能填 Java 侧名字。答案版 url:`jdbc:mysql://127.0.0.1:3306/library?useSSL=false&characterEncoding=UTF-8&serverTimezone=Asia/Shanghai`。
6. **手写 Json 里含竖线/换行破协议**(某本书标题带 `|` 或描述换行,整行响应被误拆)→ 列表数据一律走 `Json.encode`(它把 `\ | = 换行 FS RS` 全转义),不要手工字符串拼接列表。

> 附:**越权隐患**——读者客户端窗口里根本没有"图书管理/借阅记录"入口,但这只是"藏了按钮";只要有人照 `Protocol` 直接发 `BOOK_DELETE` 报文,服务端当前会照做。**这不是安全,是漏洞**。见下"越权要亲手验"。

# 系统测试(测试岗按 `templates/测试用例表.md`)

四类必测:**冒烟**(主流程走通)/ **边界**(异常输入)/ **越权**(角色隔离)/ **并发**(抢占同一本书)。逐条测,不一致记缺陷表交对应 Owner。

## AI 辅助生成测试用例

```
这是图书借还与管理模块的协议契约。生成一份测试用例清单,覆盖:正常流程(登录→查书→借→我的借阅→还/续借→管理员看记录)、异常输入(借库存0的书/还已还记录/续已还记录/达上限再借/新增图书库存>馆藏)、越权(读者发管理命令 BOOK_DELETE/READER_LIST)、并发(两人抢借同一本 stock=1 的书)。按"编号/场景/步骤/预期"列表。
【边界】只生成测试用例表,不改代码。
```

## 越权和并发要亲手验(最重要)

- **越权**:开一个客户端用读者 `zhangsan` 登录(读者窗口没有管理入口),但**测试岗手工构造报文**(或临时改客户端)发 `BOOK_DELETE|1` / `READER_LIST` / `BORROW_RECORD|0`——**答案版当前会照常执行**,这就是越权漏洞。补法:在 `ClientHandler` 处理这些管理命令前,先校验当前连接登录用户的 `role='admin'`,否则返回 `ERR|无权限`。**客户端藏了入口不等于安全,服务端必须拦**。(要拦就需要让登录后在该连接内记住 userId/role——可在 `ClientHandler` 里保存登录态,把这条作为环节 6 的一次"加权限"局部调整练习。)
- **并发**:把《万历十五年》(初始 `stock=1`)当靶子,开 2~3 个客户端分别用 `zhangsan` / `lisi` 登录,**同时**点"借这本书"。预期:只 1 人提示借阅成功,其余提示"库存不足,借阅失败"——靠 `UPDATE t_book SET stock=stock-1 WHERE id=? AND stock>0` 按影响行数判定,**绝不会超借**。

# ✅ 环节 5 验收 checklist

- [ ] 冒烟主流程全绿(登录→查书→借书→我的借阅→还书/续借→管理员图书增删改→借阅记录/超期)
- [ ] 边界用例返回正确的 `ERR|原因`(库存不足/达上限/重复还/续已还/库存>馆藏)
- [ ] 越权:读者发管理命令的问题已暴露并(在环节6)补上服务端 role 校验
- [ ] 并发:抢借 stock=1 的书只 1 人成功
- [ ] 中文不乱码(url `characterEncoding=UTF-8`)、无粘包/半包
- [ ] 缺陷都登记 + 修复 + 回归

# 环节 6 · 局部功能调整 / 迭代

> **本环节主责**:各人改自己模块;组长把关不破坏协议;测试回归。 **定位**:效果出来后想改个小地方,一让 AI 改就"牵一发动全身"。这里教"精准改一处"。**完整场景库见 `templates/局部调整场景提示词库.md`(9 个可复制场景),这里给方法。**

## 局部调整黄金三步(改任何东西都先走)

1. **定位**:先问 AI"这功能涉及哪些文件",别急着让它改。
2. **说清现象与期望**:不说"不好看",说"借书成功后表格没刷新,期望借完自动重查列表"。
3. **只改那一处(带边界)**: `只改这一个文件/方法……不动其它,改完列出改了哪行。`

## 纠错三步法(它改错/改多了时)

定位哪里不对 → 描述现象+期望 → 只改那一处 / 撤销多余改动。口诀:**先改需求,再让 AI 改代码**;越界当场让它撤。

## 场景库(直接抄)

改 Swing 窗口样式 / 改文案 / 显示已返回但没展示的字段(如超期天数)/ 新增字段穿层(协议→服务端→客户端)/ 改一条校验(借期天数)/ 加一个筛选(只看未还)/ 改续借次数上限 / **给管理命令加服务端 role 校验(承接环节5越权)** / 修一个 bug —— 每个一段带【边界】的提示词,见 `templates/局部调整场景提示词库.md`。

## ✅ 环节 6 验收 checklist

- [ ] 每次调整前先让 AI 定位(涉及哪些文件/方法)
- [ ] 一次只改一处(想改三处分三次说)
- [ ] 改完看 AI 的改动清单,越界当场撤
- [ ] 改完跑回归(尤其越权/并发,别越改越漏)

---

# 环节 7 · 审查 · 上线 · 交付

> **本环节主责**:组长把关;文档岗产手册/PPT;测试回归。

## Step 1 · AI 代码审查(信任但验证)

```
请审查本项目全部代码,重点:① 越权(读者能否发管理命令
BOOK_DELETE/READER_LIST/BORROW_RECORD,服务端拦不拦)② SQL 注入(DAO 是不是都用
PreparedStatement 的 ? 占位,有没有拼接 SQL)③ 密码安全(是否明文存储/明文传输)④ 输入校验
(服务端 service 有没有独立校验,不只靠客户端)⑤ 借还事务与防超借对不对
(setAutoCommit/commit/rollback、原子扣库存)。按"问题/位置/风险/修复建议"列出,不要直接改代
码。
【边界】只出审查报告,不改代码。
```

> **信任度金字塔**:AI 列的问题逐条判断——高信任直接采纳(管理命令无 role 校验、密码明文该改)/中信任要核实 / 必人工的自己定;误报的驳回。不是全听 AI,这就是"信任但验证"。

# Step 2 · 打包 + 本地/局域网上线(C/S)

```
教我把这个 C/S 项目打成两个可执行 jar 并在局域网跑:
- 服务端打成 fat jar:把 MySQL JDBC 驱动一起合并进去(去掉驱动自带签名),入口设为服务端监控窗口
主类
- 客户端打成纯 Swing jar(不含驱动),入口设为登录窗主类
- 局域网:数据库和服务端 jar 只在一台"服务器电脑"跑,ServerSocket 绑 0.0.0.0 天然支持局域网;
客户端电脑起客户端 jar,登录窗填"服务器电脑的局域网 IP + 端口 9000"(不是数据库地址);服务器电脑
防火墙放行入站 TCP 9000
给我具体命令。
【边界】只给打包和上线步骤,不改业务代码。
```

> 参考答案版 `build-jars.sh`:`javac` 编译 → `jar cfe dist/library-client.jar client.ui.LoginFrame -C out .`(客户端)→ 把驱动解包进 stage、删签名后 `jar cfe dist/library-server.jar server.ui.ServerMonitorFrame ...`(服务端 fat jar,约 2.5MB 含驱动)。 **上线关键点(C/S 特有,和 B/S 不同)**:
>
> - **先起服务端,再起客户端**。服务端双击 fat jar 会弹"服务端监控窗口"(显示监听端口、在线客户端数、实时日志、启动/停止按钮),不再是"双击没反应"。
> - **客户端登录窗填的是"服务端机器的 IP",不是数据库地址**——数据库只有服务端一台连,客户端从头到尾不碰数据库。单机填 `127.0.0.1`,局域网填服务器电脑 IPv4(Windows `ipconfig`、Mac `ifconfig | grep 192`)。
> - 防火墙放行入站 TCP 9000;两台电脑在同一局域网(同路由/热点)。
> - 想要无界面后台模式:`java -cp "out:lib/mysql-connector-j-8.0.33.jar" server.LibraryServer`(控制台模式)。

# Step 3 · 交付清单(文档岗主产)

- [ ] 源码(common + server + client 三段)
- [ ] 数据库建库 + 种子 SQL(`sql/init.sql`)
- [ ] 部署文档(环境/建库/打 jar/启动顺序/局域网访问/防火墙)
- [ ] 用户手册(截图 + 操作说明:读者端借还、管理员端管书)
- [ ] 演示视频(务必录一段"多客户端并发借同一本书只 1 人成功")
- [ ] 答辩 PPT(用 `../B-答辩评分表.xlsx` 五维准备)
- [ ] 过程文档(Agent PRD / 数据字典 / 通信协议契约)——体现方法论,答辩加分

## Step 4 · 答辩

五维评分见 `../B-答辩评分表.xlsx`(功能 / Vibe 方法 / 审查安全 / 上线 / 协作)。每人讲**自己负责的模块** + **Agent 管理心得**(怎么写 PRD、怎么审查 AI、遇坑怎么用纠错三步法)。C/S 项目答辩高频问:三段式怎么通信(`Protocol` + `ClientHandler` + `Api`)、多客户端并发借书怎么防超借(`BorrowService.borrow()` 的原子 UPDATE + 事务)、服务端多线程怎么做(`LibraryServer` 每 `accept()` 开一个 `ClientHandler` 线程)。

## ✅ 环节 7 验收 checklist

- [ ] AI 审查跑过,真问题修了(尤其管理命令 role 校验/密码/服务端校验)
- [ ] 两个 jar 都能起,服务端先起、客户端能连、局域网另一台电脑能用
- [ ] 交付清单齐
- [ ] 每人能讲清自己模块 + Agent 管理心得

---

# 配套模板库(`templates/`,已全建齐)

| 模板 | 用途 | 对应环节 |
|---|---|---|
| `分工看板模板.md` | 角色/命令组/窗口/文件归属/集成节奏/冻结登记 | 0 |
| `数据字典模板.md` | 逐表字段 + 库存×借阅状态机 + 评审 checklist | 2 |
| `通信协议契约模板.md` | 全局约定 + 逐命令格式 + 评审 checklist + Owner 认领 | 3 |
| `测试用例表.md` | 冒烟/边界/越权/并发 + 缺陷记录 | 5 |
| `局部调整场景提示词库.md` | 9 个局部调整场景可复制提示词 | 6 |

> **答案版工程**:`05-projects/cs-swing-library-demo/`(共享 common + 服务端 server + 客户端 client 三段式,建库 SQL + 两个可执行 jar),环节 2-5 全程跑通验证过,可作讲师演示/学生对照。⚠️ 仅供对照学习,直接抄交按学术不端处理。